Systèmes d'exploitation — concepts fondamentaux

Informatique — Licence 1 (L1)

Cours INF-L1-08

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Comprendre ce que fait réellement le logiciel invisible qui démarre avec chaque ordinateur : gérer les processus, partager la mémoire, organiser les fichiers et offrir des interfaces aux utilisateurs.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Rôle et fonctions d'un système d'exploitation
• Leçon 2 — Gestion des processus et ordonnancement
• Leçon 3 — Gestion de la mémoire centrale
• Leçon 4 — Systèmes de fichiers et arborescences
• Leçon 5 — Interfaces utilisateur : graphiques et ligne de commande
• Leçon 6 — Comparaison de Windows, Linux et macOS
• Leçon 7 — Installation et configuration de base d'un système
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Chaque fois que tu allumes un ordinateur ou un téléphone, un premier programme prend les commandes avant tous les autres : le système d'exploitation. C'est lui qui affiche le bureau, lance tes applications, sauvegarde tes fichiers et répartit le travail entre le processeur, la mémoire et les périphériques. Sans lui, chaque application devrait savoir piloter elle-même chaque modèle d'écran, de disque ou d'imprimante : un cauchemar de complexité qui rendrait l'informatique moderne impossible. On le rencontre partout sans le remarquer : Windows sur les PC, macOS chez Apple, Linux sur les serveurs et jusque dans Android sur des milliards de téléphones. Apprendre comment il fonctionne, c'est comprendre ce qui se passe réellement entre le moment où tu cliques et celui où la machine répond.

Ce cours dévoile les mécanismes fondamentaux cachés derrière cette apparente simplicité. Tu y découvriras comment le système donne l'illusion que plusieurs programmes travaillent en même temps alors que le processeur n'exécute qu'une instruction à la fois, comment il découpe la mémoire pour que les programmes ne se marchent pas dessus, et comment il organise le disque en arborescences de dossiers et de fichiers protégés par des droits d'accès. Tu apprendras aussi à calculer, avec les algorithmes d'ordonnancement, le temps qu'un programme passe réellement à attendre son tour, et à comparer les grandes familles de systèmes — Windows, Linux, macOS — sans idées reçues, jusqu'à savoir installer et configurer le tien proprement.

Ces notions ne sont pas de la théorie abstraite : elles expliquent pourquoi un ordinateur « rame », pourquoi un programme « plante », ou pourquoi un serveur Linux

peut héberger des centaines de sites à la fois. Elles fondent aussi la suite de ta formation, de l'administration système à la programmation système, et elles font partie des connaissances que les recruteurs vérifient dès les premiers entretiens techniques. Tout au long du cours, les exemples s'appuient sur des calculs simples que tu pourras refaire à la main — diagrammes de Gantt, traductions d'adresses, lectures de droits d'accès — et sur de vraies commandes à tester sur ta machine, pour que chaque concept reste attaché à un geste concret.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Définir un système d'exploitation et expliquer son rôle d'intermédiaire entre l'utilisateur, les applications et le matériel.
- Décrire les états d'un processus et le rôle du bloc de contrôle (PCB).
- Appliquer les algorithmes d'ordonnancement FCFS, SJF et Round Robin et calculer leurs temps d'attente moyens.
- Expliquer comment la mémoire centrale est partagée entre programmes grâce à la pagination et à la mémoire virtuelle.
- Naviguer dans une arborescence de fichiers, distinguer chemins absolus et relatifs et interpréter les droits d'accès.
- Comparer les grandes familles de systèmes (Windows, Linux, macOS) selon leurs principes, leurs usages et leurs licences.
- Décrire les étapes d'une installation et réaliser une configuration de base sûre d'un système.

Leçon 1 — Rôle et fonctions d'un système d'exploitation

Un système d'exploitation (SE, ou OS en anglais pour Operating System) est l'ensemble de programmes qui s'intercale entre le matériel de l'ordinateur et les applications utilisées par l'utilisateur. Quand un traitement de texte veut enregistrer un document, il ne parle pas directement au disque dur : il demande le service au système, qui se charge des détails matériels. Le système joue ainsi le rôle d'un interprète et d'un chef d'orchestre : il reçoit les demandes de tous les programmes et les convertit en opérations sur le processeur, la mémoire, le disque, l'écran ou le réseau.

Ses fonctions se regroupent en quatre grands chantiers. D'abord, la gestion des processus : décider quel programme utilise le processeur, et à quel moment. Ensuite, la gestion de la mémoire : attribuer à chaque programme un espace de travail et empêcher qu'il empiète sur celui des voisins. Puis la gestion des fichiers : organiser le stockage de manière durable et retrouver chaque document à la demande. Enfin, la gestion des entrées-sorties et de la sécurité : piloter les périphériques grâce à des pilotes (drivers) et contrôler qui a le droit de faire quoi grâce aux comptes utilisateurs et aux permissions.

Au cœur du système se trouve le noyau (kernel en anglais) : c'est le programme chargé en mémoire dès le démarrage, qui reste résident en permanence et réalise les fonctions vitales. Autour de lui gravitent les programmes système (utilitaires, services, interface graphique) qui rendent la machine agréable à utiliser. Cette organisation en couches — matériel, noyau, programmes système, applications, utilisateur — explique une règle d'or : une application qui plante ne doit jamais entraîner tout le système avec elle, car le noyau garde le contrôle de la machine.

L'essentiel
Le système d'exploitation est l'intermédiaire obligatoire entre les applications et le matériel. Ses quatre fonctions principales : gérer les processus, la mémoire, les fichiers et les entrées-sorties, le tout orchestré par le noyau, toujours présent en mémoire.

Exemple — Les couches d'un ordinateur en fonctionnement

```
+-----------------------------------------+
| Utilisateur |
+-----------------------------------------+
| Applications (navigateur, traitement |
| de texte, jeux, logiciels de gestion...) |
+-----------------------------------------+
| SYSTEME D'EXPLOITATION |
| noyau : processus, memoire, fichiers, |
| entrees-sorties, securite |
| + interface (bureau graphique ou shell) |
+-----------------------------------------+
| Materiel : processeur, memoire vive (RAM),|
| disque, ecran, clavier, carte reseau |
+-----------------------------------------+

Exemple de trajet d'une demande :
1. Tu cliques sur « Enregistrer » dans ton editeur.
2. L'application demande au noyau d'ecrire le fichier.
3. Le noyau commande le pilote du disque.
4. Le disque stocke les donnees ; le noyau confirme.
```

À retenir

- Le système d'exploitation abstrait le matériel : les applications demandent des services, elles ne pilotent pas les composants.
- Quatre fonctions clés : processus, mémoire, fichiers, entrées-sorties (avec la sécurité en fil rouge).
- Le noyau est le cœur toujours résident du système ; il est chargé au démarrage et garde le contrôle de la machine.

Exercice

Pendant une journée d'utilisation de ton ordinateur, cite quatre actions concrètes (ouvrir un document, écouter de la musique, imprimer un devoir, te connecter au Wi-Fi) et, pour chacune, indique : (a) quelle ressource matérielle est sollicitée, (b) quelle fonction du système d'exploitation intervient, (c) ce que l'application aurait dû

faire elle-même si le système n'existait pas.

✔ Corrigé
Ouvrir un document : (a) disque + mémoire vive ; (b) gestion des fichiers pour retrouver le document et gestion de la mémoire pour le charger ; (c) sans système, l'application devrait connaître l'organisation physique exacte du disque et réserver elle-même un emplacement en mémoire sans gêner les autres programmes. Écouter de la musique : (a) carte son + disque ou réseau ; (b) entrées-sorties et ordonnancement pour alimenter le haut-parleur sans coupure pendant que tu travailles ; (c) l'application devrait piloter chaque modèle de carte son. Imprimer : (a) imprimante ; (b) gestion des entrées-sorties, avec une file d'attente si plusieurs documents sont envoyés ; (c) chaque logiciel devrait intégrer les pilotes de toutes les imprimantes. Wi-Fi : (a) carte réseau ; (b) pilotes et services réseau du système ; (c) l'application devrait gérer elle-même les protocoles radio. Conclusion : le système mutualise ces tâches une fois pour toutes, ce qui rend les applications plus simples et interchangeables.

Leçon 2 — Gestion des processus et ordonnancement

Un programme enregistré sur le disque est un objet inerte, comme une partition de musique. Quand le système le lance, il devient un processus : une instance en cours d'exécution, dotée de son code, de ses données, de sa pile et de l'état courant de ses registres. Un même programme peut donner naissance à plusieurs processus (ouvre trois fenêtres de navigateur : ce sont trois processus distincts). Le système attribue à chacun un identifiant unique, le PID, et range tout ce qu'il doit savoir sur lui dans un bloc de contrôle appelé PCB (Process Control Block) : état, compteur ordinal, registres sauvegardés, mémoire allouée, fichiers ouverts.

Au cours de sa vie, un processus traverse plusieurs états. Nouveau : il vient d'être créé. Prêt : il attend son tour pour utiliser le processeur. Élu (en exécution) : le processeur exécute ses instructions. Bloqué : il attend un événement extérieur, par exemple la fin d'une lecture sur le disque ou une frappe au clavier. Terminé : il a fini son travail et le système récupère ses ressources. Comme il n'y a souvent qu'un seul processeur (ou un nombre limité de cœurs) pour des dizaines de processus prêts, le système effectue sans cesse des commutations de contexte : il sauvegarde l'état du processus sortant dans son PCB et restaure celui de l'entrant, en quelques microsecondes, donnant l'illusion d'un travail simultané.

Choisir quel processus prêt passe en exécution, c'est le travail de l'ordonnanceur, selon des algorithmes aux philosophies différentes. FCFS (premier arrivé, premier servi) exécute les processus dans leur ordre d'arrivée, jusqu'au bout : simple et équitable en apparence, mais un gros calcul peut faire attendre tous les autres. SJF (le plus court d'abord) choisit le processus dont la durée restante est la plus faible : il minimise le temps d'attente moyen, mais exige de connaître les durées à l'avance et peut affamer les longs processus. Round Robin (tourniquet) distribue le processeur par tranches de temps égales, le quantum : chacun avance à tour de rôle, ce qui garantit un temps de réponse correct aux systèmes interactifs, au prix de nombreuses

commutations.

L'essentiel
Un processus est un programme en cours d'exécution, décrit par son PCB et identifié par un PID. Il passe par les états nouveau, prêt, élu, bloqué, terminé. L'ordonnanceur choisit le prochain élu : FCFS suit l'ordre d'arrivée, SJF favorise les plus courts, Round Robin partage le temps en quanta.

Exemple — Ordonnancement FCFS : calcul complet

```
Quatre processus arrivent a l'instant 0, dans l'ordre
P1, P2, P3, P4. Durees d'execution (unites de temps) :
P1 = 8 P2 = 4 P3 = 2 P4 = 6

Diagramme de Gantt en FCFS (chacun va jusqu'au bout) :
| P1 | P2 | P3 | P4 |
0 8 12 14 20

Temps d'attente = instant de debut - instant d'arrivee :
P1 : 0 - 0 = 0
P2 : 8 - 0 = 8
P3 : 12 - 0 = 12
P4 : 14 - 0 = 14
Attente moyenne = (0 + 8 + 12 + 14) / 4 = 34 / 4 = 8,5

Observation : P3, qui n'a besoin que de 2 unites,
attend 12 unites a cause des processus places
avant lui. C'est la faiblesse classique de FCFS.
```

À retenir

- Programme ≠ processus : le programme est le fichier, le processus est son exécution vivante avec son contexte (PCB, PID).
- Un processus bloqué n'utilise pas le processeur : il attend un événement, ce qui libère le processeur pour les autres.
- FCFS est simple mais peut pénaliser les petits travaux ; SJF minimise l'attente moyenne ; Round Robin favorise la réactivité interactive.

Exercice

Reprends exactement les quatre processus de l'exemple (P1 = 8, P2 = 4, P3 = 2, P4 = 6, tous arrivés à l'instant 0), mais ordonnance-les avec SJF non préemptif : à chaque fois que le processeur se libère, on choisit le processus prêt dont la durée totale est la plus courte. Dessine le diagramme de Gantt, calcule le temps d'attente de chaque processus et le temps d'attente moyen. Compare avec le résultat FCFS de l'exemple (8,5) et explique la différence.

✔ Corrigé
À l'instant 0, les quatre processus sont prêts : le plus court est P3 (2). Il s'exécute de 0 à 2. À l'instant 2, il reste P1 (8), P2 (4), P4 (6) : le plus court est P2 (4), exécuté de 2 à 6. À l'instant 6, il reste P1 (8) et P4 (6) : P4 s'exécute de 6 à 12. P1 termine de 12 à 20. Diagramme : | P3 | P2 | P4 | P1 | avec les bornes 0, 2, 6, 12, 20. Attentes (début − arrivée, arrivée = 0 pour tous) : P3 : 0, P2 : 2, P4 : 6, P1 : 12. Attente moyenne = (0 + 2 + 6 + 12) / 4 = 20 / 4 = 5. C'est nettement mieux que les 8,5 de FCFS : en traitant d'abord les petits travaux, on évite qu'ils patientent derrière un long calcul. En contrepartie, SJF suppose de connaître (ou d'estimer) les durées à l'avance, et un flux continu de petits processus pourrait repousser indéfiniment un gros processus : c'est le risque de famine, que les systèmes réels corrigent en vieillissant artificiellement les processus qui attendent trop.

Leçon 3 — Gestion de la mémoire centrale

La mémoire centrale (la RAM) est l'espace de travail du processeur : rapide, mais limitée et volatile — tout s'efface quand la machine s'éteint. Le système doit y loger le noyau, les processus en cours et leurs données, en garantissant deux choses : que chacun dispose de la place nécessaire, et qu'aucun ne puisse lire ou modifier la mémoire d'un autre, volontairement ou par erreur. Cette protection est fondamentale : sans elle, un simple bug dans un jeu pourrait détruire le travail de ton traitement de texte, et un programme malveillant lirait tes mots de passe directement chez ses voisins.

La technique dominante est la pagination. La mémoire est découpée en cadres de taille fixe (souvent 4 Ko), et l'espace d'adressage de chaque processus est découpé en pages de la même taille. Une page peut être placée dans n'importe quel cadre libre : plus besoin de trouver un grand trou d'un seul tenant, ce qui supprime presque toute la fragmentation externe — ce gaspillage où la mémoire libre totale suffit, mais dispersée en morceaux trop petits. Une table des pages, tenue par le système, fait la correspondance entre les pages du processus et les cadres physiques ; le processeur la consulte à chaque accès grâce à un composant matériel, la MMU.

La pagination permet un tour de force : la mémoire virtuelle. Le système peut faire croire à chaque processus qu'il dispose d'un espace immense et continu, alors que seule une partie de ses pages est réellement en RAM ; les autres dorment sur le disque, dans une zone d'échange (le swap). Quand le processeur réclame une page absente, il se produit un défaut de page : le système suspend le processus, va chercher la page sur le disque, la charge dans un cadre libre (en expulsant éventuellement une page peu utilisée), puis relance l'instruction interrompue. Si la RAM est vraiment trop petite pour l'usage, les défauts de page se multiplient et la machine passe son temps à échanger avec le disque : c'est l'écroulement (thrashing), cette lenteur extrême bien connue des ordinateurs surchargés.

L'essentiel
La RAM est partagée et protégée par le système. La pagination découpe mémoire et processus en blocs de même taille (pages et cadres) reliés par une table des pages. La mémoire virtuelle prolonge la RAM sur le disque (swap) : les pages absentes sont chargées à la demande, au prix d'un défaut de page.

Exemple — Pagination : traduire une adresse, pas à pas

```
Taille de page : 4 Ko = 4096 octets.
Toute adresse logique se decompose en :
(numero de page, deplacement dans la page)

Adresse logique a traduire : 12 500
- Numero de page = 12 500 / 4096 = 3
(car 3 x 4096 = 12 288 et 4 x 4096 = 16 384)
- Deplacement = 12 500 - 12 288 = 212

La table des pages du processus dit :
page 3 -> cadre 7

Adresse physique = 7 x 4096 + 212
= 28 672 + 212
= 28 884

Le deplacement (212) ne change jamais pendant
la traduction : seule la page devient un cadre.
```

À retenir

- Chaque processus voit un espace d'adressage propre et continu, même si ses pages sont éparpillées dans la RAM.
- Une adresse = (numéro de page, déplacement) ; la table des pages convertit la page en cadre, le déplacement reste identique.
- Le swap prolonge la RAM sur le disque, mais un excès de défauts de page provoque l'écroulement : la machine échange au lieu de travailler.

Exercice

Un ordinateur possède 32 Ko de mémoire vive et des pages de 4 Ko. Un processus occupe 26 Ko. 1) Combien de cadres compte la mémoire physique ? 2) Combien de pages faut-il au processus ? 3) Quelle quantité de mémoire est perdue à l'intérieur de la dernière page du processus (fragmentation interne) ? 4) Le système peut-il encore charger entièrement un deuxième processus de 10 Ko ? Justifie.

✔ Corrigé
1) Nombre de cadres = 32 Ko / 4 Ko = 8 cadres. 2) Le processus de 26 Ko demande 26 / 4 = 6,5 pages, donc 7 pages : la dernière n'est remplie qu'à moitié (il faut 2 Ko sur les 4 Ko de la page). 3) Fragmentation interne = 4 Ko − 2 Ko = 2 Ko perdus dans la dernière page. C'est le prix, modeste et borné par la taille d'une page, de la pagination. 4) Après le premier processus, il reste 8 − 7 = 1 cadre libre, soit 4 Ko : impossible d'y loger entièrement les 10 Ko (qui demanderaient 3 pages). En revanche, grâce à la mémoire virtuelle, le deuxième processus peut démarrer quand même : le système charge les pages les plus utiles et laisse les autres dans le swap, quitte à les faire venir au fur et à mesure — ou il expulse des pages peu utilisées du premier processus. La pagination ne supprime pas le manque de mémoire, mais elle permet de le gérer finement au lieu d'exiger de grands blocs contigus.

Leçon 4 — Systèmes de fichiers et arborescences

Un fichier est un ensemble nommé d'informations conservées durablement sur un support de stockage : texte, image, programme, musique… Le système de fichiers est la partie du système d'exploitation qui organise cette conservation : il décide comment les données sont disposées sur le disque, tient le catalogue de leurs emplacements et offre les opérations de base — créer, lire, écrire, renommer, déplacer, supprimer. Chaque fichier possède des métadonnées entretenues par le système : son nom, sa taille, ses dates de création et de modification, son propriétaire et ses droits d'accès.

Pour s'y retrouver parmi des milliers de fichiers, on les range dans des répertoires (ou dossiers), qui peuvent eux-mêmes contenir d'autres répertoires : on obtient une arborescence dont la racine est le sommet unique. Sous Linux et macOS, la racine s'écrit « / » et tout descend d'elle ; sous Windows, chaque lecteur (C:, D:) possède sa propre arborescence. Pour désigner un fichier, on écrit son chemin : absolu, depuis la racine (/home/etudiant/cours/notes.txt), ou relatif, depuis le répertoire courant (cours/notes.txt si l'on se trouve dans /home/etudiant). Deux raccourcis universels : « . » désigne le répertoire courant et « .. » son parent.

Sur les systèmes de type Unix (Linux, macOS), la sécurité des fichiers repose sur trois catégories d'acteurs — le propriétaire, son groupe et les autres — et trois droits élémentaires : lecture (r), écriture (w) et exécution (x). On les lit d'un coup d'œil avec la commande ls -l : par exemple -rwxr-xr-- signifie un fichier ordinaire que le propriétaire peut lire, écrire et exécuter, le groupe lire et exécuter, et les autres seulement lire. En notation octale, r vaut 4, w vaut 2 et x vaut 1 : ce même fichier porte le mode 754, que l'on applique avec chmod 754. Pour un répertoire, les droits changent de sens : r permet de lister son contenu, w d'y créer ou supprimer des fichiers, x de le traverser pour accéder à ce qu'il contient.

L'essentiel
Le système de fichiers organise le stockage durable en une arborescence de répertoires et de fichiers, repérés par des chemins absolus (depuis la racine) ou relatifs (depuis le répertoire courant). Sous Unix, chaque fichier porte des droits r/w/x pour le propriétaire, le groupe et les autres.

Exemple — Une arborescence et ses chemins (Linux)

```
/ <- racine
+-- home/
| +-- etudiant/ <- repertoire personnel
| +-- cours/
| | +-- notes_SE.txt
| | +-- td1.pdf
| +-- photos/
| +-- plage.jpg
+-- etc/ <- reglages du systeme
+-- usr/ <- programmes installes

Chemins absolus (depuis la racine) :
/home/etudiant/cours/notes_SE.txt
/home/etudiant/photos/plage.jpg

Je suis dans /home/etudiant/cours.
Chemins relatifs vers les memes fichiers :
notes_SE.txt (ici meme)
../photos/plage.jpg (.. = remonter a etudiant)
```

À retenir

- Arborescence : une racine unique, des répertoires qui contiennent fichiers et sous-répertoires.
- Chemin absolu = depuis la racine ; chemin relatif = depuis là où je suis ; « . » = ici, « .. » = le parent.
- Droits Unix : r (lire) = 4, w (écrire) = 2, x (exécuter/traverser) = 1, donnés dans l'ordre propriétaire, groupe, autres.

Exercice

1) La commande ls -l affiche pour un fichier : -rw-r----- 1 awa etudiants 2048 notes.txt. Qui peut lire ce fichier ? Qui peut le modifier ? L'utilisateur fatou, qui n'est ni awa ni membre du groupe etudiants, peut-elle l'ouvrir ? 2) Écris le mode octal correspondant à ces droits. 3) On veut ensuite que le groupe puisse aussi modifier le fichier, sans rien changer pour les autres : quelle commande chmod faut-il exécuter (en octal) ?

✔ Corrigé

1) Les droits se lisent par triplets après le premier caractère (- = fichier ordinaire) : propriétaire awa : rw- (lecture et écriture, pas d'exécution) ; groupe etudiants : r-- (lecture seule) ; autres : --- (aucun droit). Donc awa peut lire et modifier, les membres du groupe etudiants peuvent seulement lire, et fatou, classée dans « autres », ne peut ni ouvrir ni même lire le fichier : le système lui refusera l'accès. 2) Propriétaire rw- = 4 + 2 + 0 = 6 ; groupe r-- = 4 ; autres --- = 0 : le mode est 640. 3) Donner l'écriture au groupe fait passer son triplet de r-- à rw-, soit 6 : le nouveau mode est 660, appliqué par chmod 660 notes.txt. Remarque : seul le propriétaire du fichier (ou l'administrateur) a le droit de changer ses permissions.

Leçon 5 — Interfaces utilisateur : graphiques et ligne de commande

L'interface est la partie visible du système : c'est par elle que l'utilisateur envoie ses ordres et reçoit les résultats. L'interface graphique (GUI) repose sur des fenêtres, des icônes, des menus et la souris — le modèle « WIMP » popularisé par les ordinateurs personnels. Son grand avantage est la découverte : on explore les possibilités en regardant les menus, sans rien avoir à mémoriser. Le bureau de Windows, l'environnement GNOME de Linux ou celui de macOS appartiennent à cette famille, tout comme les écrans tactiles des téléphones, qui en sont une variante adaptée aux doigts.

L'interface en ligne de commande (CLI) fonctionne à l'opposé : on tape des ordres écrits dans un terminal, interprétés par un programme appelé shell (Bash ou Zsh sous Linux/macOS, PowerShell ou l'invite de commandes sous Windows). La première prise de contact est rude — écran noir, aucune indication, tout repose sur la mémoire des commandes — mais la puissance est incomparable : on peut enchaîner les commandes automatiquement dans des scripts, combiner leur sortie grâce aux tubes (le symbole | envoie le résultat d'une commande dans la suivante), traiter mille fichiers en une ligne là où le clic demanderait une journée, et administrer à distance des serveurs qui, souvent, n'ont même pas d'écran.

Il ne s'agit pas de choisir un camp définitif : les deux interfaces parlent au même noyau et leurs qualités sont complémentaires. L'interface graphique excelle pour les tâches occasionnelles et visuelles (retouche d'image, navigation dans les dossiers), la ligne de commande pour les tâches répétitives, massives ou distantes, où la précision et l'automatisation comptent. La trajectoire d'un informaticien est presque toujours la même : commencer par le graphique, puis découvrir que le terminal devient indispensable dès qu'il faut installer des outils, diagnostiquer un problème ou gérer un serveur.

L'essentiel
Deux façons de commander le même système : l'interface graphique, intuitive et visuelle, et la ligne de commande (shell dans un terminal), austère mais précise, scriptable et automatisable. Les professionnels combinent les deux selon la tâche.

Exemple — Premières commandes dans un terminal Linux

```
$ pwd
/home/etudiant
$ ls
cours photos telechargements
$ mkdir projets
$ cd projets
$ pwd
/home/etudiant/projets
$ echo "Bonjour IAM" > salut.txt
$ cat salut.txt
Bonjour IAM
$ ls -l
-rw-r--r-- 1 etudiant etudiant 12 salut.txt
$ cp salut.txt copie.txt
$ mv copie.txt ../photos/
$ cd ..
$ ls photos
copie.txt
```

À retenir

- GUI : on montre et on clique — idéale pour découvrir et pour les tâches visuelles.
- CLI : on écrit des ordres au shell — idéale pour répéter, automatiser (scripts), combiner (tubes |) et travailler à distance.
- pwd, ls, cd, mkdir, cat, cp, mv, rm forment le kit de survie du terminal ; man <commande> affiche l'aide complète.

Exercice

Tu es connecté à un terminal Linux et ton répertoire courant est /home/etudiant, qui contient déjà un dossier cours. Écris la suite exacte de commandes pour : 1) créer un dossier revisions dans cours ; 2) te placer dans ce nouveau dossier ; 3) y créer un fichier vide fiche1.txt ; 4) afficher un récapitulatif (le chemin courant puis la liste détaillée du contenu). Attention aux chemins : choisis entre chemins relatifs et absolus et justifie ton choix.

✔ Corrigé
Depuis /home/etudiant, les chemins relatifs sont les plus courts : 1) mkdir cours/revisions crée le dossier en descendant directement dans cours. 2) cd cours/revisions te place dedans. 3) touch fiche1.txt crée le fichier vide dans le répertoire courant. 4) pwd affiche /home/etudiant/cours/revisions, puis ls -l montre fiche1.txt avec ses droits et sa taille (0 octet). L'alternative en chemins absolus (mkdir /home/etudiant/cours/revisions, etc.) fonctionne aussi, mais elle est plus longue à taper et casse si le dossier personnel change de nom ; le chemin relatif est préférable tant qu'on connaît sa position de départ, que pwd permet justement de vérifier avant de commencer.

Leçon 6 — Comparaison de Windows, Linux et macOS

Windows, développé par Microsoft, équipe la grande majorité des ordinateurs personnels dans le monde. Sa force historique est la compatibilité : presque tous les logiciels grand public et professionnels, ainsi que les jeux vidéo, paraissent d'abord pour lui, et il reconnaît une immense variété de matériels. C'est un système propriétaire : sa licence est payante (souvent incluse dans le prix de l'ordinateur) et son code source est fermé. Son interface graphique est son seul visage officiel, même si PowerShell offre une ligne de commande de plus en plus capable. Sa popularité en fait aussi la cible principale des virus, ce qui impose une vigilance constante sur les mises à jour de sécurité.

Linux est d'une nature très différente : c'est un noyau libre et gratuit, créé en 1991 par Linus Torvalds et amélioré depuis par une communauté mondiale, autour duquel on assemble des distributions complètes — Ubuntu, Debian, Fedora, Linux Mint — qui ajoutent interface, logiciels et outils d'installation. Le code source est ouvert : chacun peut l'étudier, le modifier et le redistribuer. Cette liberté, la stabilité du système et sa sobriété en ressources expliquent sa domination écrasante sur les serveurs, les supercalculateurs et les systèmes embarqués ; Android, d'ailleurs, repose sur un noyau Linux. Sur l'ordinateur personnel, il demande un petit effort d'adaptation, récompensé par l'absence de licence à payer et un contrôle total sur sa machine.

macOS, le système d'Apple, est un système propriétaire réservé aux ordinateurs de la marque : on ne peut pas l'installer légalement sur un PC ordinaire. Issu d'une base Unix, il partage avec Linux de nombreuses commandes de terminal, ce qui en fait un environnement apprécié des développeurs. Son identité repose sur l'intégration poussée entre matériel et logiciel et sur un écosystème cohérent avec l'iPhone et l'iPad. En résumé, le choix d'un système dépend du contexte : Windows pour la compatibilité maximale avec les logiciels du marché, Linux pour la liberté, les serveurs et l'apprentissage de l'informatique en profondeur, macOS pour l'expérience intégrée de l'univers Apple — un informaticien gagne à savoir passer de l'un à l'autre, car les concepts de ce cours sont valables pour tous.

L'essentiel
Windows : propriétaire, payant, compatibilité logicielle maximale, cible n°1 des malwares. Linux : noyau libre et gratuit, distributions variées, roi des serveurs et base d'Android. macOS : propriétaire, lié au matériel Apple, fondations Unix. Les concepts (processus, mémoire, fichiers) restent identiques partout.

Exemple — Tableau comparatif synthétique

```
Critere Windows Linux macOS
------------- ------------- ------------- -------------
Editeur Microsoft Communaute + Apple
entreprises
Licence Proprietaire, Libre et Proprietaire,
payante gratuite lie au Mac
Code source Ferme Ouvert Ferme
Materiel PC tres PC, serveurs, Mac
varies embarque uniquement
Interface Graphique Graphique Graphique
(+PowerShell) + terminal + terminal
tres puissant Unix
Usage dominant Postes de Serveurs, Postes
bureau, jeux supercalculs, creatifs,
Android ecosysteme
Apple
```

À retenir

• Libre ≠ gratuit par hasard : Linux est modifiable et redistribuable légalement, ce qui explique ses mille distributions.

• La part de marché ne dit pas tout : Linux est minoritaire sur les bureaux mais majoritaire sur les serveurs et dans Android.

• Changer de système ne change pas les concepts : processus, mémoire, fichiers et droits se retrouvent partout.

Exercice

Pour chacun des trois profils suivants, recommande un système d'exploitation et justifie en deux ou trois arguments tirés de la leçon : (a) une petite entreprise de Bamako qui doit équiper cinq postes de bureautique avec un budget très serré ; (b) un étudiant dont le programme de formation exige un logiciel de comptabilité qui n'existe que sous Windows ; (c) un développeur qui veut apprendre l'administration de serveurs et héberger ses propres projets.

✔ Corrigé
(a) Linux (par exemple Linux Mint ou Ubuntu) : aucune licence à payer pour les cinq postes, besoins en bureautique couverts par des suites libres comme LibreOffice, et consommation modérée de ressources qui prolonge la vie de machines modestes ; il faudra prévoir une courte formation des utilisateurs. (b) Windows : le critère décisif est la compatibilité logicielle — le logiciel imposé ne fonctionne que là, et un poste de travail doit d'abord faire tourner les outils du métier ; la licence est le coût d'accès à cet écosystème. (c) Linux également, mais pour d'autres raisons : c'est le système des serveurs réels, le terminal y est central, et tout ce qu'il apprendra (droits, processus, services) est directement transférable à un hébergement professionnel ; il peut l'installer gratuitement, y compris dans une machine virtuelle. Conclusion : il n'y a pas de « meilleur » système en absolu, il y a un système adapté à un usage, un budget et des contraintes logicielles.

Leçon 7 — Installation et configuration de base d'un système

Installer un système d'exploitation consiste à le copier sur le disque dur de la machine et à le rendre capable de démarrer seul. On peut le faire sur un disque vierge, en remplacement du système existant, ou à côté de lui : c'est le double démarrage (dual boot), qui propose un choix au démarrage — pratique pour garder Windows et découvrir Linux sur le même ordinateur. L'installation démarre depuis un support externe, typiquement une clé USB préparée avec l'image ISO du système ; l'ordinateur doit être réglé pour démarrer sur cette clé, ce qui se règle dans le micrologiciel de la machine (BIOS ou UEFI) ou par un menu de démarrage ponctuel.

Le programme d'installation guide ensuite l'utilisateur à travers quelques décisions importantes. Le partitionnement divise le disque en zones logiques : une partition système, éventuellement une partition séparée pour les données personnelles (/home sous Linux), et la partition d'échange (swap). Viennent ensuite le fuseau horaire, la langue du clavier — piège classique des installations francophones — et la création du premier compte utilisateur, qui doit être distingué du compte administrateur (root sous Linux, protégé par sudo) : au quotidien, on travaille avec un compte ordinaire et l'on n'élève ses privilèges que pour les opérations d'administration, ce qui limite les dégâts d'une fausse manipulation comme d'un logiciel malveillant.

Un système fraîchement installé n'est pas encore prêt à travailler : la configuration de base commence. On connecte la machine au réseau, on lance immédiatement les mises à jour de sécurité, on installe les pilotes manquants éventuels et les logiciels nécessaires, puis on met en place une stratégie de sauvegarde — car aucun disque n'est éternel. Deux réflexes protègent presque tout : sauvegarder ses données AVANT toute installation (le partitionnement mal compris est la première cause de pertes de fichiers chez les débutants) et vérifier que la machine répond aux exigences minimales du système choisi. Le TP qui suit ce cours te fera pratiquer une installation complète, sans risque, dans une machine virtuelle.

L'essentiel
Installer = démarrer sur un support d'installation (clé USB ISO), partitionner le disque, créer un compte ordinaire distinct de l'administrateur, puis configurer : réseau, mises à jour de sécurité, pilotes, logiciels et sauvegardes. Toujours sauvegarder ses données avant de toucher aux partitions.

Exemple — Les grandes étapes d'une installation (exemple Linux)

```
AVANT
1. Sauvegarder ses documents sur un support externe
2. Verifier la configuration minimale requise
3. Telecharger l'image ISO officielle du systeme
4. Creer une cle USB demarrable a partir de l'ISO

DEMARRAGE
5. Demarrer l'ordinateur sur la cle USB
(menu de boot du BIOS/UEFI, souvent touche F12)
6. Choisir « Essayer ou installer » -> Installer

INSTALLATION GUIDEE
7. Langue + disposition du clavier (francais !)
8. Partitionnement : effacer le disque OU
installer a cote (dual boot) OU manuel
9. Fuseau horaire (ex. : Afrique/Bamako)
10. Creer l'utilisateur + mot de passe solide

APRES LE REDEMARRAGE
11. Se connecter, regler le reseau (Wi-Fi/cable)
12. Lancer TOUTES les mises a jour de securite
13. Installer ses logiciels, restaurer ses donnees
14. Activer une sauvegarde reguliere
```

À retenir

- Le dual boot fait cohabiter deux systèmes sur un même disque : un menu au démarrage laisse choisir.
- Compte ordinaire pour le quotidien, privilèges administrateur (sudo/root) seulement pour administrer : c'est une règle de sécurité, pas une contrainte administrative.
- Données d'abord : aucune installation ne se lance sans sauvegarde préalable vérifiée.

Exercice

Un camarade veut installer Linux en dual boot à côté de Windows pour suivre ce cours, sans perdre ni son système ni ses documents. Il te dit : « Je lance l'installateur, je clique partout sur Suivant, et on verra bien. » Liste, dans l'ordre, les précautions qu'il doit prendre avant de commencer, les choix critiques pendant l'installation, et les trois premières actions à faire après le premier démarrage. Explique pour chaque point le risque évité.

✔ Corrigé
Avant : sauvegarder tous ses documents sur un disque externe ou une clé (risque évité : la perte totale en cas d'erreur de partitionnement) ; vérifier l'espace disque libre et la configuration minimale (évite une installation qui échoue ou un système inutilisable) ; télécharger l'ISO uniquement depuis le site officiel de la distribution et créer la clé USB démarrable (évite les images modifiées contenant des malwares) ; noter que ses fichiers Windows doivent rester accessibles, donc ne rien formater à l'aveuglette. Pendant : choisir l'option « installer à côté de Windows » plutôt que « effacer le disque » (c'est LE choix critique : la seconde détruit Windows et ses données) ; vérifier la disposition du clavier français avant de taper le mot de passe (sinon le mot de passe saisi ne sera pas celui attendu aux connexions suivantes) ; créer un compte utilisateur ordinaire avec un mot de passe solide et différent. Après : se connecter au réseau ; lancer l'ensemble des mises à jour de sécurité avant toute autre installation ; vérifier que Windows démarre toujours depuis le menu de boot, puis installer ses logiciels de travail et seulement ensuite recopier ses données. Suivre cet ordre transforme une opération risquée en procédure maîtrisée.

Travaux dirigés (TD)

TD 1 — Round Robin : diagramme et temps d'attente

Trois processus arrivent tous à l'instant 0 et sont rangés dans la file des prêts dans l'ordre P1, P2, P3. Durées d'exécution : P1 = 5, P2 = 3, P3 = 1 unité de temps. L'ordonnanceur applique le tourniquet (Round Robin) avec un quantum de 2 unités : un processus qui n'a pas terminé après 2 unités retourne en fin de file. 1) Établis le diagramme de Gantt complet jusqu'à la fin du dernier processus. 2) Donne l'instant de terminaison de chaque processus. 3) Calcule le temps d'attente de chacun (rappel : attente = instant de terminaison − instant d'arrivée − durée d'exécution) et le temps d'attente moyen. 4) Compare avec FCFS appliqué aux mêmes processus et commente.

✔ Corrigé du TD 1
1) Déroulement, quantum = 2. File initiale : [P1, P2, P3]. De 0 à 2 : P1 travaille, il lui reste 5 − 2 = 3, il retourne en file → [P2, P3, P1]. De 2 à 4 : P2 travaille, il lui reste 3 − 2 = 1 → file [P3, P1, P2]. De 4 à 5 : P3 n'a besoin que d'1 unité, il termine à l'instant 5 (il rend le processeur avant la fin de son quantum) → file [P1, P2]. De 5 à 7 : P1 travaille, il lui reste 3 − 2 = 1 → file [P2, P1]. De 7 à 8 : P2 consomme sa dernière unité et termine à 8 → file [P1]. De 8 à 9 : P1 termine à 9. Gantt : | P1 | P2 | P3 | P1 | P2 | P1 | bornes 0, 2, 4, 5, 7, 8, 9. 2) Terminaisons : P3 à 5, P2 à 8, P1 à 9. 3) Attentes : P1 = 9 − 0 − 5 = 4 ; P2 = 8 − 0 − 3 = 5 ; P3 = 5 − 0 − 1 = 4. Attente moyenne = (4 + 5 + 4) / 3 = 13 / 3 ≈ 4,33. 4) En FCFS (ordre P1, P2, P3) : exécutions 0–5, 5–8, 8–9 ; attentes : P1 = 0, P2 = 5, P3 = 8, moyenne = 13 / 3 ≈ 4,33 aussi. Les moyennes sont ici identiques, mais la répartition diffère : en Round Robin, P3 (le petit travail) termine dès l'instant 5 au lieu de 9 et aucun processus n'attend plus de 5 unités avant de commencer (temps de première réponse : 0, 2 et 4 contre 0, 5 et 8 en FCFS). C'est exactement ce qu'on demande à un système interactif : que chaque utilisateur voie sa tâche démarrer vite, même si la moyenne globale ne s'améliore pas.

TD 2 — Arborescence, chemins et droits : manipulations guidées

L'étudiante Awa travaille sous Linux, son répertoire personnel est /home/awa. Elle exécute la suite de commandes suivante : cd /home/awa ; mkdir -p IAM/cours/SE ; cd IAM/cours ; touch notes.txt ; cp notes.txt SE/ ; cd SE ; pwd. 1) Dessine l'arborescence complète créée sous /home/awa. 2) Que contient chaque répertoire à la fin ? 3) Que affiche la dernière commande pwd ? 4) Donne le chemin absolu du fichier copié et son chemin relatif depuis /home/awa/IAM. 5) ls -l sur la copie affiche -rw-r--r-- : traduis ces droits en octal et dis si un autre étudiant peut modifier cette copie.

✔ Corrigé du TD 2

1) Arborescence : /home/awa contient IAM/ ; IAM contient cours/ ; cours contient notes.txt et SE/ ; SE contient notes.txt (la copie). 2) À la fin : cours/ contient le fichier original notes.txt et le sous-répertoire SE ; SE/ contient la copie notes.txt. Remarque : cp copie, donc l'original reste en place. 3) Après cd IAM/cours puis cd SE, le répertoire courant est /home/awa/IAM/cours/SE : c'est ce qu'affiche pwd. 4) Chemin absolu de la copie : /home/awa/IAM/cours/SE/notes.txt. Chemin relatif depuis /home/awa/IAM : cours/SE/notes.txt. 5) -rw-r--r-- : propriétaire rw- = 6, groupe r-- = 4, autres r-- = 4, soit le mode 644. Un autre étudiant (catégorie « autres », ou même membre du groupe) dispose seulement du droit de lecture : il peut ouvrir et lire la copie, mais pas la modifier — toute tentative d'écriture sera refusée par le système, et seul awa (ou l'administrateur) peut changer ces droits avec chmod.

Travaux pratiques (TP)

TP 1 — Observer les processus et la mémoire de sa propre machine

Objectif : vérifier concrètement, sur une machine réelle, les notions des leçons 2 et 3 (processus, états, mémoire). Sous Linux, ouvre un terminal et exécute : 1) ps aux | head pour lister les processus en cours ; repère la colonne STAT et identifie au moins un processus en état R (élu/en cours) et des processus en état S (endormi, en attente d'événement). 2) Lance la commande sleep 60 en arrière-plan (sleep 60 &), retrouve son PID avec ps, observe-la dans top, puis termine-la proprement avec kill <PID>. 3) Affiche l'état de la mémoire avec free -h : relève la mémoire totale, utilisée, libre et la taille du swap. 4) Recommence les relevés 1 et 3 sous Windows (Gestionnaire des tâches, onglets Processus et Performance) ou sous macOS (Moniteur d'activité) si tu disposes de ces systèmes, et compare les informations présentées. Livrables : un compte rendu d'une page avec tes relevés, les PID observés et tes réponses : combien de processus tournaient au repos ? Que se passe-t-il dans top quand tu lances sleep ?

✔ Attendus du TP 1
Attendus : la liste ps contient des dizaines, voire des centaines de processus (démons du système, shell, interface graphique) alors même que « rien ne semble ouvert » : c'est la multiprogrammation permanente. Dans STAT, la majorité des processus sont en S (ils attendent un événement : c'est l'état bloqué/endormi de la leçon 2) et un petit nombre en R, ce qui illustre que le processeur ne sert à chaque instant qu'une minorité d'entre eux. Le processus sleep apparaît avec un PID propre ; dans top, il consomme 0 % de processeur car il est bloqué en attente du temps qui passe ; kill <PID> le fait passer à l'état terminé et il disparaît des listes. free -h montre qu'une partie de la RAM est occupée par les caches du système (colonne buff/cache) — ce n'est pas un gaspillage : le système prête la mémoire libre au cache disque et la reprend dès qu'un processus en a besoin — et la ligne swap indique la taille de la zone d'échange de la mémoire virtuelle. Dans le Gestionnaire des tâches Windows, on retrouve les mêmes informations sous une forme graphique (processus, % processeur, mémoire), preuve que les concepts sont identiques d'un système à l'autre. Erreurs fréquentes à éviter : tuer un processus système au hasard (ne termine que TON sleep), confondre PID et nom du programme, et conclure « mémoire presque pleine = problème » alors que les caches gonflent volontairement les chiffres.

Exercices corrigés supplémentaires

Exercice 1

Ordonnancement par priorité (non préemptif). Trois processus arrivent à l'instant 0 : P1 (durée 6, priorité 2), P2 (durée 3, priorité 1), P3 (durée 4, priorité 3). Convention : plus le numéro est petit, plus le processus est prioritaire. À chaque libération du processeur, l'ordonnanceur choisit le processus prêt le plus prioritaire, qui s'exécute jusqu'au bout. Établis le diagramme de Gantt et le temps d'attente moyen.

✔ Corrigé
À l'instant 0, les trois processus sont prêts : le plus prioritaire est P2 (priorité 1), exécuté de 0 à 3. À l'instant 3, il reste P1 (priorité 2) et P3 (priorité 3) : P1 s'exécute de 3 à 9. P3 termine de 9 à 13. Gantt : | P2 | P1 | P3 | bornes 0, 3, 9, 13. Attentes (début d'exécution, arrivée = 0) : P2 = 0, P1 = 3, P3 = 9. Attente moyenne = (0 + 3 + 9) / 3 = 12 / 3 = 4. Discussion : l'ordonnancement par priorité sert à favoriser les tâches importantes (par exemple les processus du système avant les tâches de fond), mais il partage avec SJF le risque de famine : si des processus prioritaires arrivent sans cesse, les processus de faible priorité peuvent attendre indéfiniment. Les systèmes réels ajoutent un mécanisme de vieillissement : plus un processus attend, plus sa priorité effective augmente, jusqu'à obtenir forcément le processeur.

Exercice 2

Chemins relatifs. Tu te trouves dans le répertoire /home/etudiant/cours/SE de la machine de la leçon 4 (arborescence de l'exemple). Sans utiliser de chemin absolu, écris le chemin relatif qui désigne : 1) le fichier notes_SE.txt situé dans le même répertoire que td1.pdf ; 2) le fichier plage.jpg ; 3) le répertoire etc. Pour chaque réponse, décompose le trajet (remonter, descendre).

✔ Corrigé
Position de départ : /home/etudiant/cours/SE. 1) notes_SE.txt se trouve dans cours, c'est-à-dire le répertoire parent : il faut remonter d'un cran (..) puis désigner le fichier : ../notes_SE.txt. 2) plage.jpg se trouve dans /home/etudiant/photos : remonter deux fois (de SE vers cours, puis vers etudiant), descendre dans photos, nommer le fichier : ../../photos/plage.jpg. 3) etc est un répertoire de la racine. Le chemin de départ compte quatre éléments après la racine (home, etudiant, cours, SE), il faut donc quatre « .. » pour remonter jusqu'à / : le premier ramène dans cours, le deuxième dans etudiant, le troisième dans home et le quatrième à la racine. Le chemin relatif vers /etc s'écrit alors ../../../../etc. Leçon de l'exercice : au-delà de deux ou trois remontées, le chemin absolu /etc est plus court et beaucoup moins source d'erreurs que la version relative.

Exercice 3

Défaut de page et mémoire virtuelle : scénario à analyser. Un ordinateur de 4 Go de RAM fait tourner en même temps un navigateur avec de nombreux onglets, un traitement de texte et un jeu. Soudain, tout devient très lent et le disque dur « gratte » en permanence. 1) Explique le phénomène en mobilisant les notions de pages, de swap et de défaut de page. 2) Cite deux solutions immédiates sans dépenser d'argent. 3) Cite la solution matérielle définitive et explique pourquoi elle supprime le phénomène.

✔ Corrigé
1) La mémoire demandée par l'ensemble des processus dépasse les 4 Go de RAM : le système a donc déchargé dans le swap les pages les moins utilisées. Mais navigateur, traitement de texte et jeu sont tous actifs : dès que l'utilisateur bascule d'un programme à l'autre, les pages nécessaires sont absentes de la RAM — défaut de page — et le système doit les relire sur le disque, des milliers de fois par minute. Le temps du processeur est alors consumé en transferts disque au lieu de servir les programmes : c'est l'écroulement (thrashing), dont le bruit de disque incessant est le symptôme caractéristique. 2) Solutions gratuites : fermer les onglets et les programmes inutilisés pour ramener l'ensemble de travail sous la barre des 4 Go ; renoncer à faire tourner le jeu en même temps que les applications lourdes, ou réduire la mémoire consommée (par exemple fermer les applications en arrière-plan). 3) Solution matérielle : ajouter de la RAM (passer à 8 ou 16 Go). Le phénomène disparaît parce que l'ensemble des pages réellement utilisées tient alors en mémoire centrale : les défauts de page deviennent rares et le swap ne sert plus qu'en cas de pic exceptionnel, exactement le rôle pour lequel il est conçu.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quelle est la fonction première d'un système d'exploitation ?

A) Remplacer les applications comme le traitement de texte

B) Servir d'intermédiaire entre l'utilisateur, les applications et le matériel

C) Accélérer la connexion Internet

2. Comment s'appelle le cœur du système d'exploitation, chargé au démarrage et toujours présent en mémoire ?

A) Le noyau (kernel)

B) Le pilote (driver)

C) Le shell

3. Un processus qui attend la fin d'une lecture sur disque est dans l'état…

A) Élu

B) Prêt

C) Bloqué

4. Dans l'algorithme FCFS, le processeur est attribué…

A) Au processus le plus court

B) Dans l'ordre d'arrivée des processus

C) Par tranches de temps égales

5. Le quantum est la tranche de temps caractéristique de quel algorithme ?

A) Round Robin (tourniquet)

B) SJF

C) FCFS

6. La mémoire virtuelle permet…

A) De doubler physiquement la RAM installée

B) D'effacer les virus présents en mémoire

C) De faire fonctionner des programmes plus gros que la RAM en utilisant le disque (swap)

7. Dans la pagination, une adresse logique est traduite grâce à…

A) La table des pages, qui associe chaque page à un cadre de la RAM

B) Le registre d'horloge du processeur

C) Le nom du fichier exécutable

8. Lequel de ces chemins est un chemin absolu ?

A) /home/etudiant/cours/notes.txt

B) ../photos/plage.jpg

C) cours/notes.txt

9. Le mode 755 appliqué à un fichier signifie…

A) Lecture seule pour tout le monde

B) Aucun accès pour le groupe et les autres

C) Propriétaire : lecture/écriture/exécution ; groupe et autres : lecture/exécution

10. Quel est l'avantage décisif de la ligne de commande sur l'interface graphique ?

A) Elle affiche de plus belles couleurs

B) Elle permet d'automatiser les tâches par des scripts et de traiter des milliers de fichiers d'un coup

C) Elle ne nécessite aucun apprentissage

Corrigés du QCM

1. Réponse B — Le système reçoit les demandes des applications et les traduit en opérations sur le matériel : c'est un intermédiaire et un gestionnaire de ressources.

2. Réponse A — Le noyau réalise les fonctions vitales (processus, mémoire, fichiers, entrées-sorties) et reste résident en mémoire pendant toute la session.

3. Réponse C — Bloqué : il attend un événement extérieur et ne peut pas utiliser le processeur, même si celui-ci est libre.

4. Réponse B — FCFS = First Come, First Served : premier arrivé, premier servi, chaque processus allant jusqu'au bout de son exécution.

5. Réponse A — En Round Robin, chaque processus reçoit le processeur pour au plus un quantum, puis retourne en fin de file des prêts.

6. Réponse C — Les pages peu utilisées sont déchargées sur le disque ; le processus dispose d'un espace d'adressage plus grand que la mémoire physique.

7. Réponse A — Page → cadre via la table des pages ; le déplacement à l'intérieur de la page reste inchangé.

8. Réponse A — Un chemin absolu commence à la racine (le caractère / en tête sous Linux). Les deux autres sont relatifs au répertoire courant.

9. Réponse C — 7 = 4+2+1 (rwx) pour le propriétaire ; 5 = 4+1 (r-x) pour le groupe et pour les autres.

10. Réponse B — Scripts, tubes et traitement en masse : c'est la puissance de la CLI, en contrepartie d'un apprentissage initial des commandes.

Résumé — les notions importantes

• Le système d'exploitation est l'intermédiaire entre applications et matériel ; son noyau, toujours en mémoire, gère processus, mémoire, fichiers et entrées-sorties.

• Un processus est un programme en cours d'exécution, identifié par un PID et décrit par son PCB ; il traverse les états nouveau, prêt, élu, bloqué et terminé.

• Ordonnancement : FCFS suit l'ordre d'arrivée, SJF choisit le plus court (attente moyenne minimale), Round Robin partage le processeur en quanta pour garantir la réactivité.

• La pagination découpe la mémoire en pages et cadres de même taille : fini les grands blocs contigus, la table des pages traduit les adresses.

• La mémoire virtuelle prolonge la RAM par le swap sur disque ; trop de défauts de page entraînent l'écroulement, quand la machine passe son temps à échanger.

• Les fichiers vivent dans une arborescence de répertoires ; on les désigne par chemin absolu ou relatif, et les droits r/w/x (propriétaire, groupe, autres) contrôlent leur accès.

• Windows, Linux et macOS incarnent trois philosophies (propriétaire compatible, libre et ouvert, intégré à un écosystème) ; installer un système exige sauvegarde préalable, partitionnement réfléchi et mises à jour immédiates.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Rôle et fonctions d'un système d'exploitation
Le système d'exploitation abstrait le matériel : les applications demandent des services, elles ne pilotent pas les composants. ; Quatre fonctions clés : processus, mémoire, fichiers, entrées-sorties (avec la sécurité en fil rouge). ; Le noyau est le cœur toujours résident du système ; il est chargé au démarrage et garde le contrôle de la machine.

Fiche 2 — Gestion des processus et ordonnancement
Programme ≠ processus : le programme est le fichier, le processus est son exécution vivante avec son contexte (PCB, PID). ; Un processus bloqué n'utilise pas le processeur : il attend un événement, ce qui libère le processeur pour les autres. ; FCFS est simple mais peut pénaliser les petits travaux ; SJF minimise l'attente moyenne ; Round Robin favorise la réactivité interactive.

Fiche 3 — Gestion de la mémoire centrale
Chaque processus voit un espace d'adressage propre et continu, même si ses pages sont éparpillées dans la RAM. ; Une adresse = (numéro de page, déplacement) ; la table des pages convertit la page en cadre, le déplacement reste identique. ; Le swap prolonge la RAM sur le disque, mais un excès de défauts de page provoque l'écroulement : la machine échange au lieu de travailler.

Fiche 4 — Systèmes de fichiers et arborescences
Arborescence : une racine unique, des répertoires qui contiennent fichiers et sous-répertoires. ; Chemin absolu = depuis la racine ; chemin relatif = depuis là où je suis ; « . » = ici, « .. » = le parent. ; Droits Unix : r (lire) = 4, w (écrire) = 2, x (exécuter/traverser) = 1, donnés dans l'ordre propriétaire, groupe, autres.

Fiche 5 — Interfaces utilisateur : graphiques et ligne de commande
GUI : on montre et on clique — idéale pour découvrir et pour les tâches visuelles. ; CLI : on écrit des ordres au shell — idéale pour répéter, automatiser (scripts), combiner (tubes |) et travailler à distance. ; pwd, ls, cd, mkdir, cat, cp, mv, rm forment le kit de survie du terminal ; man <commande> affiche l'aide complète.

Fiche 6 — Comparaison de Windows, Linux et macOS
Libre ≠ gratuit par hasard : Linux est modifiable et redistribuable légalement, ce qui explique ses mille distributions. ; La part de marché ne dit pas tout : Linux est minoritaire sur les bureaux mais majoritaire sur les serveurs et dans Android. ; Changer de système ne change pas les concepts : processus, mémoire, fichiers et droits se retrouvent partout.

Fiche 7 — Installation et configuration de base d'un système
Le dual boot fait cohabiter deux systèmes sur un même disque : un menu au démarrage laisse choisir. ; Compte ordinaire pour le quotidien, privilèges administrateur (sudo/root) seulement pour administrer : c'est une règle de sécurité, pas une contrainte administrative. ; Données d'abord : aucune installation ne se lance sans sauvegarde préalable vérifiée.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — 01 - Introduction aux Systèmes d'Exploitation

Chaîne : professeur youssef · Langue : Français · Niveau : Débutant
Une introduction claire au rôle du système d'exploitation comme intermédiaire entre le matériel et les logiciels. Elle pose les notions de gestion des ressources et d'interface utilisateur qui ouvrent ce cours.

https://www.youtube.com/watch?v=98atyiXNXgk

Leçon 2 — Système d’exploitation : la gestion des processus – ch 1.3

Chaîne : MOOC NSI : Numérique et Sciences Informatiques · Langue : Français · Niveau : Débutant à intermédiaire
Cette vidéo de MOOC explique comment le système fait fonctionner plusieurs processus en quasi-parallèle sur un même processeur. Elle illustre directement les états et les changements de contexte de la leçon 2.

https://www.youtube.com/watch?v=PnHXpUGVWNI

Leçon 2 — Gestion des Processus - Travaux Dirigées TD

Chaîne : Cours.ComputerScience · Langue : Français · Niveau : Intermédiaire
Un TD filmé qui applique FCFS, SJF, le tourniquet (Round Robin) et les priorités sur des exercices d'ordonnancement. Parfait pour vérifier ta méthode après le TD de ce cours, calculs à l'appui.

https://www.youtube.com/watch?v=8BiLyvbspT4

Leçon 5 — Tutoriel Shell: 10 Commandes Bash pour Débutants

Chaîne : AmbreCode · Langue : Français · Niveau : Débutant
Les dix commandes de base du terminal (pwd, ls, cd, mkdir, touch, cp, mv, rm, cat, echo) démontrées une à une. À regarder terminal ouvert pour reproduire chaque

commande en même temps.

https://www.youtube.com/watch?v=2RC2ZnXMBPE

Leçon 7 — Installer Linux : les 3 méthodes en 1 vidéo (du débutant au confirmé)

Chaîne : HelloLinux · Langue : Français · Niveau : Débutant

Créer une clé USB démarrable, tester Linux dans une machine virtuelle ou l'installer en dual boot à côté de Windows : les trois voies d'installation de la leçon 7, montrées pas à pas. La méthode machine virtuelle est celle recommandée pour le TP associé à ce cours.

https://www.youtube.com/watch?v=_DzVc12Kzto

Sources et références

• Andrew Tanenbaum, « Systèmes d'exploitation » (Pearson) : l'ouvrage de référence universitaire sur les processus, la mémoire et les systèmes de fichiers.

• Abraham Silberschatz et coll., « Operating System Concepts » (Wiley) : le manuel classique des cours de systèmes, riche en exercices d'ordonnancement.

• Documentation officielle des distributions Linux (ubuntu.com, debian.org) et de Microsoft Learn (learn.microsoft.com) pour les procédures d'installation et d'administration de base.

• Le manuel intégré à tout système Unix : la commande man (par exemple « man chmod ») documente chaque commande et chaque droit, directement dans le terminal.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !